

CH4: RELIABILITY ISSUES IN DDBSS

Contents

Reliability in DDBMS	1
Failures in DDBMS.....	1
Techniques to Ensure Reliability in DDBMS.....	2
Commit Protocols	3
1. Two-Phase Commit Protocol (2PC)	3
Coordinator Timeout Scenarios	5
Participant Timeout Scenarios	5
2. Three-Phase Commit Protocol (3PC).....	6
Coordinator Timeouts	7
Participant Timeouts	7

Reliability in DDBMS

Reliability in a DDBS refers to the ability of the system to continue functioning correctly and provide consistent and correct data access even in the presence of various failures such as hardware malfunctions, communication errors, or software crashes.

- Since a DDBMS operates across multiple, networked sites, it is more vulnerable to partial failures.
- Hence, ensuring reliability is critical tekeko maintain data consistency, availability, and fault tolerance.

Failures in DDBMS

Failures in DDBMS can occur due to many reasons. These failures disrupt normal operations and can compromise data reliability if not handled properly.

a. Site Failure

- A complete site (machine or node) becomes non-operational.
- All the databases, transactions, and services running on that site become unavailable.
- Causes: Power failure, hardware crash, system crash.

b. Link Failure

- The communication link between two or more sites is broken or disrupted.
- Affects the ability of sites to coordinate and access distributed data.
- Causes: Network cable disconnection, switch/router failure, poor connectivity.

c. Transaction Failure

- An individual transaction fails due to logical errors or deadlocks.
- Only the specific transaction is affected, not the entire site.
- Causes: Programming bugs, constraint violations, deadlock, timeout.

d. Media Failure

- A storage device (like hard disk or SSD) fails, leading to loss or corruption of data.
- Affects the physical storage of data at a site.
- Causes: Hardware malfunction, disk crash, file system corruption.

e. Communication Failure

- Interruptions in message transfer between different sites or components.
- Causes inconsistent or lost transaction messages.
- Causes: Network congestion, protocol errors, or failures in communication software.

f. Coordinator Failure

- In a distributed transaction, if the site coordinating (e.g., in 2-phase commit) fails, the whole transaction may hang or become inconsistent.
- Very critical in distributed systems

Techniques to Ensure Reliability in DDBMS

a. Redundancy and Replication

- Maintain multiple copies of data (replicas) across different sites.
- If one site fails, data can be retrieved from another.
- **Types:**
 - Full Replication: All data at all sites.
 - Partial Replication: Only critical data replicated.

b. Distributed Commit Protocols

- Ensure that all sites either commit or abort a distributed transaction together.
- **Two-Phase Commit (2PC):**
 - Phase 1: Prepare – All participants agree to commit.
 - Phase 2: Commit – If all agreed, commit; else, abort.
- **Three-Phase Commit (3PC):**
 - Adds an extra phase to avoid blocking in case of coordinator failure.

c. Logging and Checkpointing

- Logs record all actions taken by transactions to allow recovery.
- Checkpoints store the state of the system periodically to minimize recovery time.
- Used for rollback and redo during recovery.

d. Failure Detection and Recovery

- Use heartbeat signals or timeouts to detect failed sites or links.
- Automatic recovery procedures restart services or reroute queries to available replicas.

e. Backup and Restore

- Regular **backups** ensure that data can be restored in case of permanent media failure.
- Useful for long-term protection and disaster recovery.

f. Quorum-based Protocols

- Ensure consistency in replicated data.
- Read and write operations must be acknowledged by a **quorum (majority) of replicas**.
- Helps in fault tolerance and preventing inconsistency.

g. Distributed Concurrency Control

- Use protocols like **Distributed Locking**, **Timestamp Ordering**, or **Optimistic Concurrency Control**.
- Prevents conflicts and ensures correctness even in the presence of concurrent transactions on different sites.

Commit Protocols

In a **Distributed Database**, a single transaction may involve **multiple sites or databases**. To ensure **atomicity** (i.e., all-or-nothing execution of the transaction), **commit protocols** are used to **coordinate the decision** (commit or abort) across all involved sites.

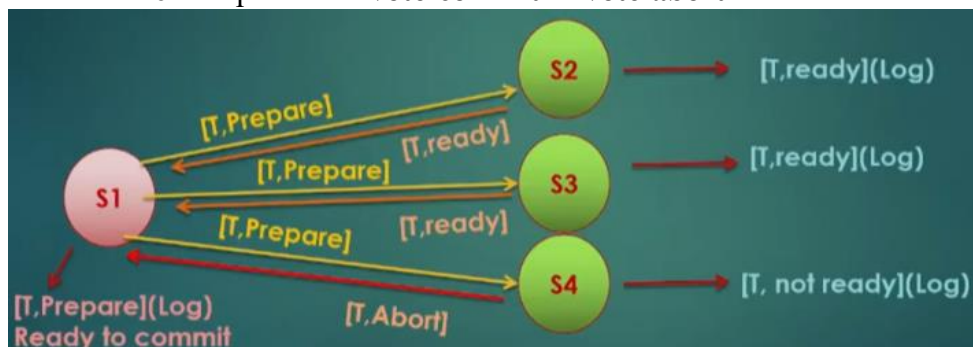
1. Two-Phase Commit Protocol (2PC)

- In distributed transaction, all its global and sub transaction should follow the property of Atomicity (all or none) to maintain consistency.
- 2 phase commit protocol ensures atomic commitment of distributed transactions.
- It checks if all site involved in the execution of the transaction are ready to commit or not.
- There are 2 modules here
 - **Coordinator**: the process at the site where the transaction originated and which controls execution
 - **Participants**: processes at the other site that participate in executing the transaction

◆ Phases of 2PC:

□ Phase 1: Voting Phase (Prepare Phase)

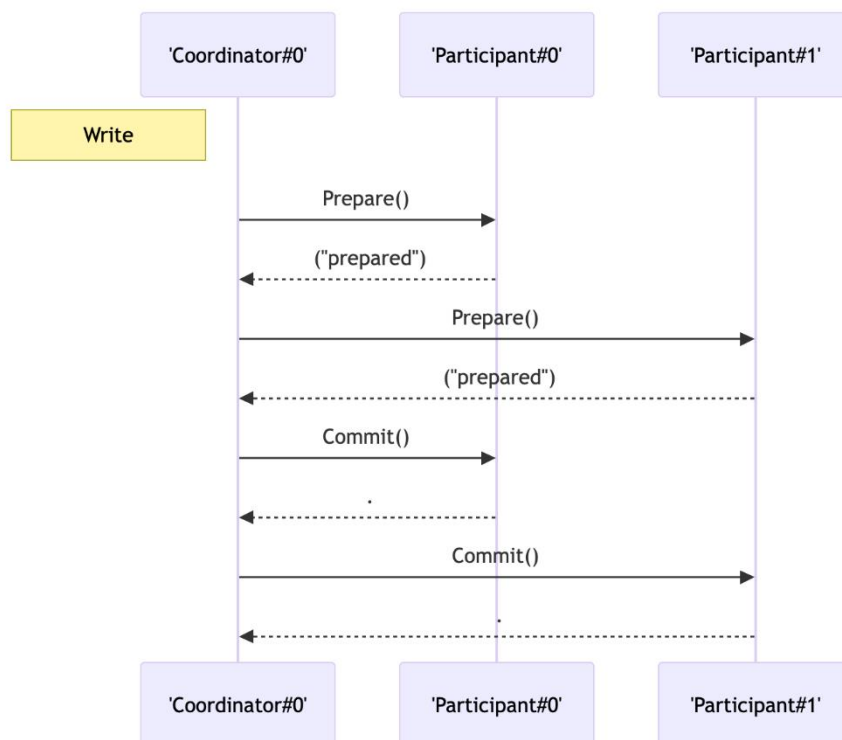
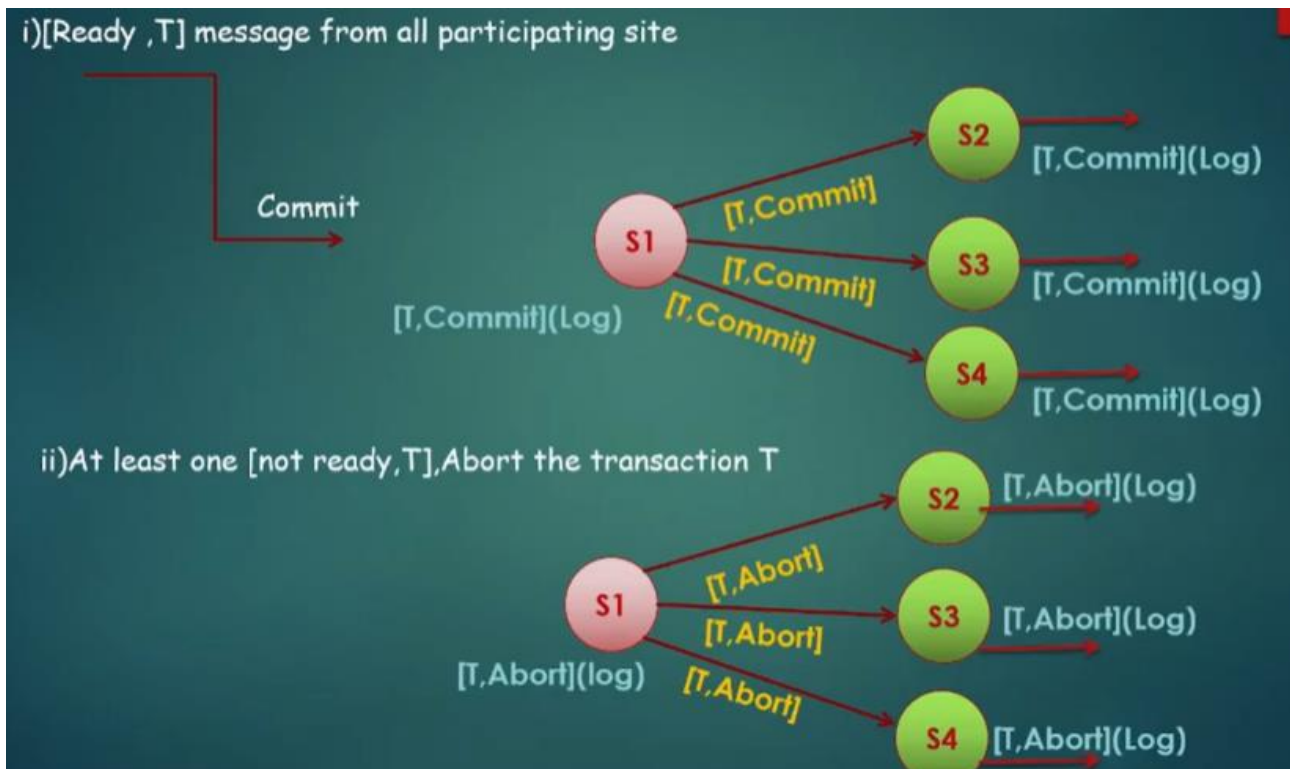
- **Coordinator** sends a *prepare* request to all participating sites (participants).
- Each participant does:
 - Writes the transaction details to its **log**.
 - Executes the transaction **up to the commit point**.
 - Replies with **vote-commit** or **vote-abort**.



As S4 is not ready, transaction would be aborted

□ Phase 2: Decision (Commit/Abort) Phase

- If **all participants vote to commit**, the coordinator sends a **global commit** message.
- If **any participant votes to abort**, the coordinator sends a **global abort** message.
- Participants then:
 - Commit or rollback the transaction.
 - Acknowledge the final outcome to the coordinator.



◆ **Advantages:**

- Simple and widely used.
- Ensures atomicity.

◆ **Disadvantages:**

- **Blocking Protocol:** If the coordinator crashes after sending the vote request but before sending the final decision, participants may be left **in doubt** and cannot proceed (i.e., they are **blocked**).
- Relies heavily on the coordinator's availability.

Coordinator Timeout Scenarios

These happen **at the coordinator's side**, usually due to a delay or failure in communication with participants.

Case 1: Coordinator Timeout While Waiting for Votes

- **Situation:** Coordinator sends Prepare request, but one or more participants **don't respond** in time.
- **Action:**
 - Coordinator assumes **those participants have failed**.
 - Aborts the transaction.
 - Sends **Global Abort** to the participants who responded.

Case 2: Coordinator Timeout After Sending Global Decision

- **Situation:** Coordinator sends Global Commit or Global Abort, but doesn't receive acknowledgments from all participants.
- **Action:**
 - Coordinator can log the decision and retry sending the decision later.
 - The decision is **already logged**, so failure recovery is safe.

Participant Timeout Scenarios

These are more critical — when participants are **waiting indefinitely**, it can cause **blocking**.

Case 1: Participant Timeout Waiting for Prepare

- **Situation:** A participant expects a Prepare request from the coordinator but doesn't receive it.
- **Action:**
 - Do nothing (passive timeout).
 - Eventually the coordinator may retry or declare abort.
 - No risk here because the participant has not done anything yet.

Case 2: Participant Timeout After Voting "Yes" (Waiting for Global Decision)

- **Situation:** Participant sends Vote-Commit but coordinator crashes before sending the decision.
- **Action:**
 - Participant is now in the **"uncertain state"** (this is the **blocking issue** of 2PC).
 - Participant **cannot decide** on its own whether to commit or abort.
 - Participant can:
 - **Wait** (until coordinator recovers),
 - **Query other participants** for the decision (if available), or
 - **Initiate termination protocol** (used in some systems).

Case 3: Participant Timeout After Sending Acknowledgment

- **Situation:** Participant sent acknowledgment after commit/abort, but doesn't hear further.
- **Action:**
 - Safe to **assume transaction is complete**.
 - Clean up local logs and resources.

Key Terms to Remember

- **Uncertain State:** When a participant has voted "Yes" and is waiting for a decision — it cannot act on its own.
- **Blocking Problem:** Occurs because participants in uncertain state cannot safely proceed if the coordinator fails.

2. Three-Phase Commit Protocol (3PC)

- **Blocking problem:** A major disadvantage of 2 PC protocol is faced when coordinator site fails and this results in blocking problem, as the participants are uncertain whether to commit or abort.
- **3PC is an extension of 2PC:** that avoids blocking even when the coordinator fails

◆ Phases of 3PC:

□ Phase 1: CanCommit Phase

- Coordinator sends a **CanCommit?** request (like "Are you ready?") to all participants.
- Participants respond with **Yes** or **No**.

□ Phase 2: PreCommit Phase

- If all respond **Yes**, the coordinator sends a **PreCommit** message.
- Participants:
 - Record a "pre-commit" log entry.
 - Prepare to commit, but do **not** actually commit yet.
 - Send an **acknowledgment** to the coordinator.

□ Phase 3: DoCommit Phase

- Once all participants have acknowledged PreCommit, the coordinator sends the **DoCommit** message.
- All participants **commit** the transaction and confirm the commit.

How Blocking problem is handled?

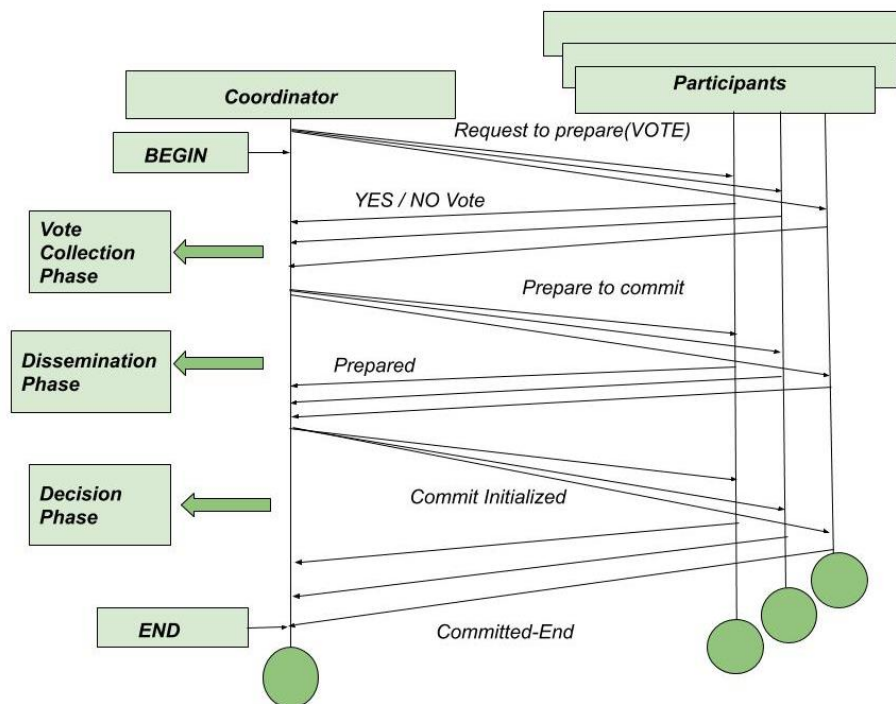
- If the coordinator fails, new coordinator is chosen which communicates with the remaining sites to check if old coordinator site had decided to commit.
- If a failure occurs, participants can use logs to decide whether to commit or abort independently.

◆ Advantages:

- **Non-blocking:** If a failure occurs, participants can use logs to decide whether to commit or abort independently.
- Better fault tolerance than 2PC.

◆ Disadvantages:

- **More overhead:** Involves an extra phase and more messages.
- Slower than 2PC in normal conditions.



Coordinator Timeouts

- ◆ **Case 1: Timeout Waiting for "Yes"/"No" from Participants (CanCommit Phase)**
 - **Action:** Coordinator aborts the transaction and sends Global Abort to responding participants.
- ◆ **Case 2: Timeout Waiting for Acks after PreCommit**
 - **Action:** Coordinator may resend PreCommit; if still no response, assume participant has failed and proceed based on logs.
- ◆ **Case 3: Timeout Waiting for Acks after DoCommit**
 - **Action:** Coordinator considers transaction committed; logs are updated. Retries sending DoCommit if needed.

Participant Timeouts

- ◆ **Case 1: Timeout Waiting for CanCommit Request**
 - **Action:** Do nothing or abort locally if no info is available.
- ◆ **Case 2: Timeout After Saying "Yes", Waiting for PreCommit**
 - **Action:** Participant assumes coordinator failed before PreCommit — **safe to abort** (since 3PC ensures no commit decision was made yet).
- ◆ **Case 3: Timeout After PreCommit, Waiting for DoCommit**
 - **Action:** Participant is in **pre-commit state** (a safe state).
 - Can communicate with other participants.
 - If majority says "PreCommit" → **commit**.
 - If others say "Abort" or "No Info" → **abort**.

Feature	Two-Phase Commit (2PC)	Three-Phase Commit (3PC)
Phases	2	3
Blocking	Yes	No
Failure Recovery	Coordinator failure can block	Can proceed using logs
Message Overhead	Fewer messages	More messages
Use Case	Simpler systems	Critical systems needing high availability